

MAE 145: Planning and Estimation

Homework #4

Assigned April 23. Due on **Apr 29**

Notice:

Note that this homework due date (problems 1, 2, and 3) is Wed April 29, 11:59pm; and the late submission is April 30, 1:00pm. The assignment has an extra-credit python programming practice (due on April 29, as always, with a late submission of May 1st), which you can choose to do as a way of obtaining extra credit points for homework. For this assignment, include the names of the other students you have collaborated with to do the homework problems. The Collaboration Policy for this course is detailed in the syllabus.

Homework exercises:

1. E3.7 from the Lecture Notes on Robotics Planning and Kinematics.
2. E3.8 from the Lecture Notes on Robotics Planning and Kinematics.
3. E4.1 from the Lecture Notes on Robotics Planning and Kinematics.
4. Optional, extra credit programming exercise (tic-tac-toe, part II). Here we continue with the tic-tac-toe problem of hw2, as a python practice. Please use the python template that summarizes the functions of part I. In the following exercises, we will write new functions to determine if a player has won the game. Do the following:
 - (a) Create a function `row_win(board, player)` that takes the `player` (integer 1 or 2) and determines if any row consists of only their marker. Have it return `True` if this condition is met and `False` otherwise. Recall that `board` is defined as in hw2, problem 3 and as a numpy 3 x 3 array consisting of 0, 1 and 2's. 0 indicates an available position, 1 indicates player 1 has chosen this position (resp. a 2 indicates player 2 has chosen this position.) Call `row_win` several times and verify that Player 1 has a complete row after several moves.
 - (b) Make a function `col_win(board, player)` that takes the `player` (integer) and determines if any column consists of only their marker. Have it return `True` if this condition is met and `False` otherwise. Call `col_win` to verify if Player 1 has a complete column after several moves.
 - (c) Create a function `diag_win(board, player)` that takes the `player` (integer) and determines if any diagonal consists of only their marker. Have it return `True` if this condition is met and `False` otherwise. Evaluate the function in two examples (for the 2 diagonals.)
 - (d) Create a function `evaluate(board)` that uses `row_win`, `col_win`, and `diag_win` functions for both players. If one of them has won, return that player's number. If the board is full but no one has won, return `-1`. Otherwise, return 0.

Note that `board` is defined as in previous exercises. Call `evaluate` to see if either player has won the game yet. Use the following sample code to get started:

```
def evaluate(board):
    winner = 0
    for player in [1, 2]:
        #add your code here!
    pass
    if np.all(board != 0) and winner == 0:
        winner = -1
    return winner
```

- (e) Now we make use of all of the functions we have created to simulate an entire game. The functions `create_board()`, `random_place(board, player)`, and `evaluate(board)` are all defined as in previous exercises. Create a function `play_game()` that:
- Creates a board.
 - Alternates taking turns between two players (beginning with Player 1), placing a marker during each turn.
 - Evaluates the board for a winner after each placement.
 - Continues the game until one player wins (returning 1 or 2 to reflect the winning player), or the game is a draw (returning -1).
 - Call `play_game` 1000 times, and store the results of the game in a list called `results`. Use `random.seed(9999999)` so we can check your answer!
- (f) From the previous exercise, you should have observed that, when guessing at random, it's better to go first, as one would expect. Let's see if Player 1 can improve their strategy. Create a function `play_strategic_game()`, where Player 1 always starts with the middle square, and otherwise both players place their markers randomly. Call `play_strategic_game` 1000 times. Set the seed 1 using `random.seed(9999999)` again so that we can check your answer.